

Implementando Padrões de Teste

Disciplina	Testes de Software
Trabalho	Implementando Padrões de Teste (Test Patterns)
Aluno(a)	Amanda Bueno Campos Peixoto

1. Introdução

O objetivo deste trabalho foi construir uma suíte de testes de unidade para o serviço CheckoutService, responsável por processar pedidos de um carrinho de compras. A solução aplicou padrões de teste para criar dados de forma clara, isolar dependências externas e validar tanto o estado retornado quanto as interações relevantes do sistema. Foram implementados o Object Mother para usuários, o Data Builder para carrinhos e Test Doubles com jest.fn() para substituir GatewayPagamento, PedidoRepository e EmailService. Os testes seguem a organização Arrange, Act, Assert, mantendo cada cenário focado em uma regra observável do checkout.

Padrão	Arquivo	Papel na suíte
Object Mother	__tests__/builders/UserMother.js	Cria usuários padrão e premium com valores previsíveis.
Data Builder	__tests__/builders/CarrinhoBuilders	Monta carrinhos flexíveis com usuário e itens específicos para cada cenário.
Stubs e Mocks	__tests__/CheckoutService.test.js	Controlam retornos externos e verificam chamadas importantes.

2. Padrões de criação de dados

2.1. Object Mother para usuários

O UserMother foi usado porque User é uma entidade simples, com poucos atributos e variações bem conhecidas. Na suíte, os métodos clientePadrao() e clientePremium() centralizam dados recorrentes, como tipo de usuário e e-mail, evitando repetição nos testes.

2.2. Por que CarrinhoBuilder em vez de CarrinhoMother

O Carrinho é mais variável do que o usuário: pode estar vazio, conter um ou vários itens, pertencer a um cliente padrão ou premium e representar totais diferentes. Se fosse usado um CarrinhoMother, seria necessário criar muitos métodos para cada combinação, como carrinhoPremiumComDoisItens(), carrinhoPadraoVazio() e carrinhoComTotalEspecifico(). Esse crescimento torna o setup mais rígido e difícil de manter.

Com o CarrinhoBuilder, cada teste declara apenas a diferença importante para o cenário. Os valores padrão ficam no construtor e os métodos fluentes, como comUsuario(), comItens() e vazio(), permitem adaptar o objeto sem esconder a intenção do teste.

2.3. Antes e depois do Builder

Antes: setup manual com muitos detalhes que não são o foco do teste

```
const user = new User(2, 'Cliente Premium',
  'cliente.premium@email.com', 'PREMIUM');
const itens = [
  new Item('Teclado', 100),
  new Item('Monitor', 200)
];
const carrinho = new Carrinho(user, itens);
```

Depois: setup expressivo, concentrado no comportamento que será validado.

```
const carrinho = new CarrinhoBuilder()
  .comUsuario(UserMother.clientePremium())
  .comItens([
    { nome: 'Teclado', preco: 100 },
    { nome: 'Monitor', preco: 200 }
  ])
  .build();
```

A versão com Builder melhora a legibilidade porque remove ruído de construção e deixa explícito que o cenário importante é um cliente premium com R\$300,00 em itens. Também melhora a manutenção: se a estrutura interna de Carrinho ou Item mudar, a alteração tende a ficar concentrada no Builder, não espalhada por todos os testes

3. Test Doubles: Stubs vs. Mocks

Dependência	Double	Uso no teste premium	Tipo de verificação
GatewayPagamento	Stub + verificação de chamada	Retorna { success: true } para simular pagamento aprovado e recebe o total com desconto.	Estado e comportamento pontual
PedidoRepository	Stub	Retorna um pedido salvo com id, evitando qualquer banco de dados real.	Estado
EmailService	Mock	Permite verificar se enviarEmail foi chamado com destinatário, assunto e corpo corretos.	Comportamento

No teste de sucesso para cliente premium, as dependências externas do CheckoutService foram substituídas por dublês de teste. A diferença principal está no motivo do uso: Stubs fornecem respostas controladas para permitir que o fluxo avance; Mocks registram chamadas para que o teste verifique interações

3.1 GatewayPagamento como Stub

O GatewayPagamento é principalmente um Stub porque o teste não deve depender de um gateway real nem de qualquer instabilidade externa. Ao configurar cobrar() para retornar { success: true }, o teste controla o fluxo e permite observar a lógica interna do serviço: cálculo do desconto, criação do pedido, salvamento e envio do e-mail.

No cenário premium também foi verificado que cobrar() recebeu R\$270,00 para um carrinho de R\$300,00. Essa checagem confirma a regra de 10% de desconto no ponto em que ela realmente impacta a integração com pagamento.

3.2 EmailService como Mock

O EmailService foi tratado como Mock porque o objetivo do teste não é obter um retorno útil do envio de e-mail, mas verificar o efeito colateral esperado após um pagamento aprovado. Por isso, o teste valida que enviarEmail() foi chamado com o e-mail do usuário, o assunto "Seu Pedido foi Aprovado!" e uma mensagem contendo o id do pedido salvo e o valor final.

Essa é uma verificação de comportamento: o resultado principal não está no objeto retornado, mas na interação correta com uma dependência externa. Em contrapartida, nos cenários de falha de pagamento, o teste garante que EmailService e PedidoRepository não são chamados, evitando efeitos colaterais indevidos.

4. Cenários testados e resultado

- Retorno null quando o GatewayPagamento recusa a cobrança.
- Pedido salvo com totalFinal correto para cliente padrão.
- Desconto de 10% aplicado para cliente PREMIUM ao chamar o gateway.
- Envio de e-mail com destinatário, assunto e corpo corretos após pagamento aprovado.
- Ausência de chamadas ao EmailService e ao PedidoRepository quando o pagamento falha.

A validação final foi executada com `npm test -- --runInBand`. O resultado obtido foi: 1 suíte de testes aprovada, 5 testes aprovados e 0 falhas.

5. Conclusão

O uso deliberado de padrões de teste ajudou a prevenir Test Smells desde a criação da suíte. O Object Mother reduziu repetição em objetos simples; o Data Builder tornou explícitas as variações relevantes do Carrinho; e os Test Doubles isolaram o CheckoutService de serviços externos. Com isso, os testes ficaram mais legíveis, rápidos, determinísticos e menos frágeis.

A separação entre Stubs e Mocks também fortaleceu a intenção de cada teste: quando o foco era o estado final, foram usados retornos controlados; quando o foco era o comportamento, foram verificadas as chamadas realizadas. Essa clareza contribui para uma suíte sustentável, capaz de evoluir junto com o código de produção sem se tornar um obstáculo para manutenção.